SE 1105 - PROGRAMMING AND PROBLEM SOLVING I

Pointers

Asst. Prof. Dr. Kazım Erdoğdu

Faculty of Engineering
Department of Software Engineering
Yaşar University

Fall, 2024

Table of Contents

1. Pointers
2. Pointers on Stack
3. Pointers on Heap

Table of Contents

1. Pointers
2. Pointers on Stack
3. Pointers on Heap

Pointers

- Where are the codes of a C program and its variables stored in memory (i.e. RAM)?
- When a C program is executed, the operating system allocates certain segments on RAM.
- On termination of a C program, the operating system frees all this space.

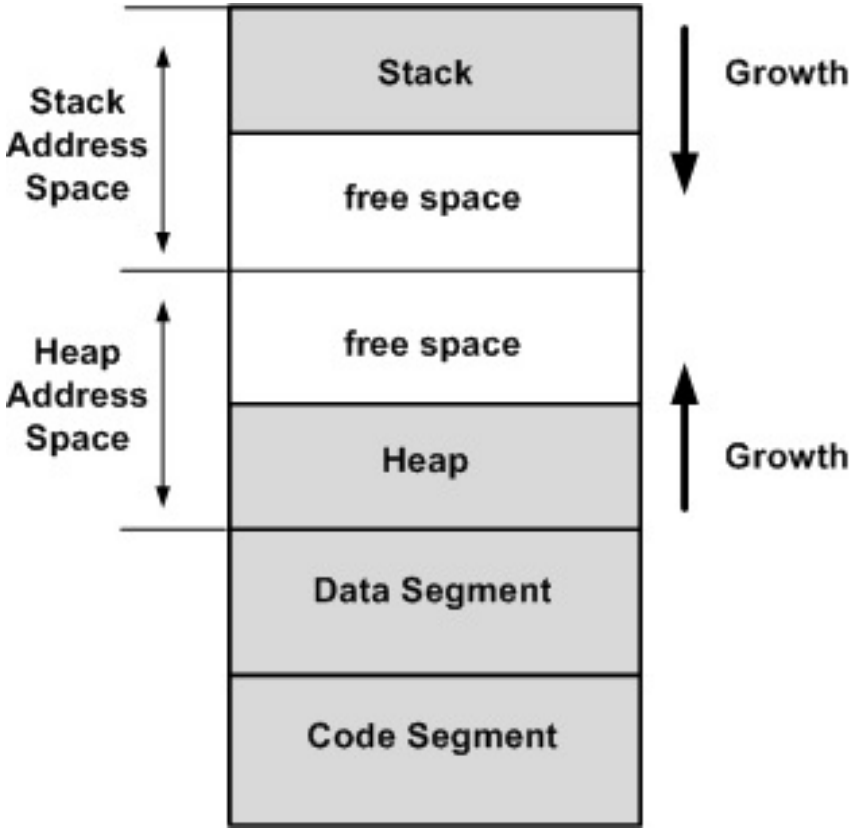


Figure: https://open4tech.com/concept-heap-usage-embedded-systems/

Pointers

- **Code Segment** contains the C programming code.
- **Data Segment** contains the global or static variables in the C program.
- **Heap** contains the dynamically allocated variables. These variables are accessed in any order.
- **Stack** contains the statically allocated local variables and function calls. These variables and function calls are accessed in LIFO (Last-In-First-Out) manner.

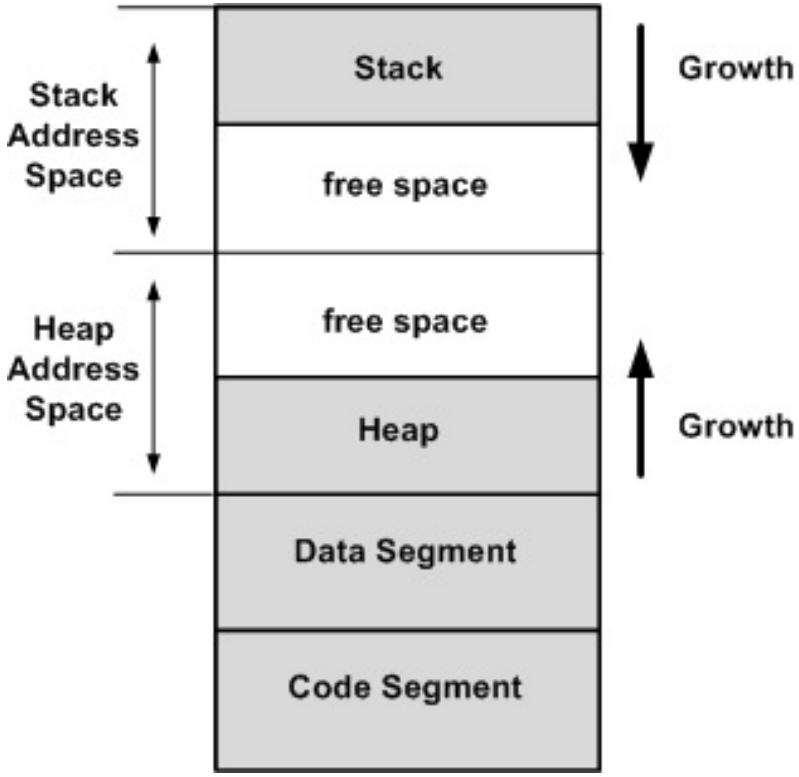


Figure: https://open4tech.com/concept-heap-usage-embedded-systems/

Table of Contents

1. Pointers
2. Pointers on Stack
3. Pointers on Heap

Stack

- Local variables are stored on the stack.
- Function calls take place on the stack.
- Stack works in a LIFO (Last-In-First-Out) manner.
- The data declared on the stack can be used without pointers.
- The variables and functions are allocated sequentially to the stack by stack frames.
- Stack allocation is faster than Heap allocation.

Stack

```c
#include <stdio.h>

int g(int x){
    return x*x;
}

int f(int a, int b){
    int c = a + g(b);
    return c;
}

void main() {
    int k = 5, m = 3;
    int n = f(k,m);
    printf("%d\n",n);
}
```

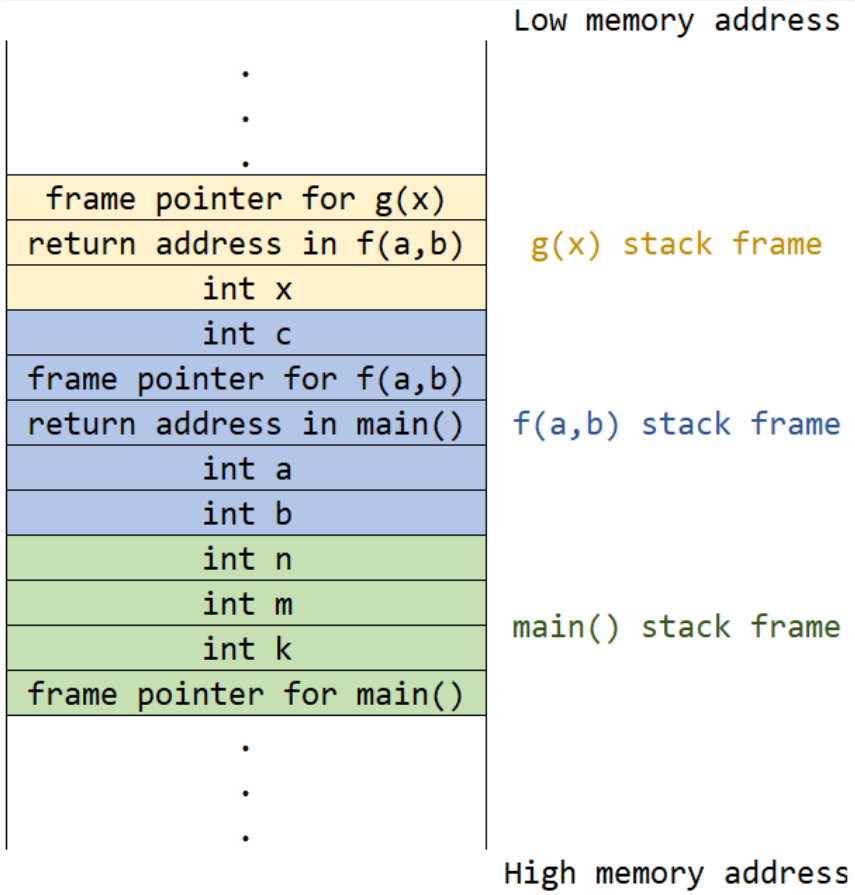


Figure: Stack

Stack

- Once a function's execution is completed, this function along with its local variables is removed from the stack. (When we say remove, there is no actual "remove" happening. It is done by stack pointer's relocation).
- It means that whatever happens with a local variable in a function stays in a function (except the return value).
- If we want to affect the values of the local variables in other functions, we need to use pointers.

Pointers

- **Pointers** are variables whose values are memory addresses (Deitel and Deitel, 2022)
- Pointers enable programs to
  - accomplish pass-by-reference,
  - call functions within functions,
  - manipulate strings and arrays,
  - declare and manipulate dynamic data structures that grow and shrink at execution time, such as linked lists, queues, stacks, and trees.

Pointers

- Pointers are declared with an asterisk (i.e. *) before the name of the pointer variable.

```
int *p;
```

- The address of a variable is retrieved by an ampersand (i.e. &). Assigning a variable's address to a pointer is called **referencing**.

```
int a = 5;
int *p = &a;
```

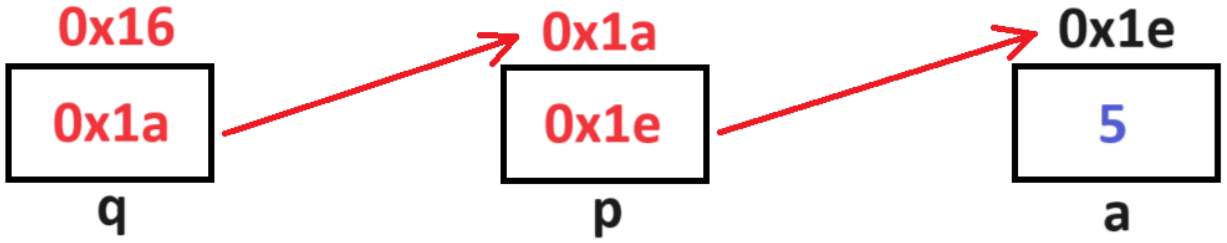
Pointers

- **%p** format specifier is used to see the address of a variable or the value in a pointer variable in hexadecimal format.
- An asterisk (i.e. *) is used before the pointer to get the value in the address that a pointer points. This is called **dereferencing**.

```
1 void main(){
2   int a = 5;
3   int *p = &a;
4   printf("a:%d, &a:%p\n",a,&a);
5   printf("*p:%d, p:%p\n",*p,p);
6 }
7 Output:
8   a:5, &a:000000dc657ffd34
9   *p:5, p:000000dc657ffd34
```

Pointers

- Pointers can also point to other pointers.




```c
void main(){
  int a = 5;
  int *p = &a;
  int **q = &p;
  printf("a:%d\n",a);
  printf("&a:%p\n",&a);
  printf("*p:%d\n",*p);
  printf("p:%p\n",p);
  printf("&p:%p\n",&p);
  printf("*q:%p\n",*q);
  printf("**q:%d\n",**q);
}
```

Output:

a:5
&a:000000bfcffffd94
*p:5
p:000000bfcffffd94
&p:000000bfcffffd88
*q:000000bfcffffd94
**q:5

Pointers

- The value at a certain address can be updated by directly manipulating the variable or manipulating the pointer's value that points to that address.

```c
void main(){
  int a = 5;
  int *p = &a;
  printf("a:%d\n",a);
  printf("*p:%d\n",*p);
  a = 7;
  printf("a:%d\n",a);
  printf("*p:%d\n",*p);
  *p = 9;
  printf("a:%d\n",a);
  printf("*p:%d\n",*p);
}
```

Output:

a:5
*p:5
a:7
*p:7
a:9
*p:9

Pointers

- Pointers have data types.
- These data types tell the pointer the size of the memory space that it will be pointing to.

```
1 void main(){
2   int a = 3077;
3   int *p = &a;
4   char *q = &a;
5   printf("a:%d\n",a);
6   printf("*p:%d\n",*p);
7   printf("*q:%d\n",*q);
8 }
```

Output:

a:3077
*p:3077
*q:5

Pointers

- As you can see from the example above, the pointer type matters.
- The pointers can be in type of **int**, **float**, **double** and **char**. *(There are some more options but these are enough for our course.)*
- There is also one more type that we can use with pointers: **void**.
- void type makes the pointer generic at definition. This pointer can be typecasted to different types later in the code.

Pointers

```
1 void main(){
2   void *p;
3   char c = 'd';
4   p = &c;
5   printf("%c\n",*(char *)p);
6   int a = 3077;
7   p = &a;
8   printf("%d\n",*(int *)p);
9 }
```

Output:

d
3077

Exercises

1. Write a swap function that receives two integers <u>pass by value</u> and observe its functionality.
2. Write a swap function that receives two integers <u>pass by reference</u> and observe its functionality.
3. Write a function that runs the following Collatz conjecture by using pointers.

$$f(n) = \begin{cases} \frac{n}{2}, & \text{n is even} \\ 3 * n + 1, & \text{otherwise} \end{cases}$$

4. Why does scanf() use & when it receives parameters?

Pointers with Arrays and Strings

- Pointers can be used to retrieve the addresses of arrays and strings.
- When we dealt with arrays and strings the pointers functioned behind the scenes. Now we will take a closer look at them.
- When we assign the address of an array or a string to a pointer, the address of the first element in them is stored by the pointer. The rest of the elements in the arrays or strings can be retrieved by pointer arithmetics.

Pointers with Arrays and Strings

```
1 void main(){
2   int A[] = {1,2,3};
3   int *p = &A;
4   for(int i=0; i<3; i++)
5       printf("&A[%d] = %p\n",
  i, &A[i]);
6   printf("&A = %p\n",&A);
7   printf("*p = %p\n",p);
8 }
```

Output:

&A[0] = 0000007730dff8b4
&A[1] = 0000007730dff8b8
&A[2] = 0000007730dff8bc
&A = 0000007730dff8b4
*p = 0000007730dff8b4

Pointers with Arrays and Strings

```
1 void main(){
2   char s[] = "SE 1105";
3   int *p = &s;
4   for(int i=0; i<7; i++)
5       printf("&s[%d] = %p\n",
  i, &s[i]);
6   printf("&s = %p\n",&s);
7   printf("*p = %p\n",p);
8 }
```

Output:

&s[0] = 000000dfb65ff848
&s[1] = 000000dfb65ff849
&s[2] = 000000dfb65ff84a
&s[3] = 000000dfb65ff84b
&s[4] = 000000dfb65ff84c
&s[5] = 000000dfb65ff84d
&s[6] = 000000dfb65ff84e
&s = 000000dfb65ff848
*p = 000000dfb65ff848

Pointer Arithmetic

- We can define the following arithmetics on pointers:
  - Incrementing or decrementing of a pointer
  - Adding or subtracting an integer from a pointer
  - Subtracting two pointers
  - Comparison of two pointers
- The first two are used for traversing arrays and strings by their addresses.
- The third one gives the number of elements of the array or string between the two pointer locations.
- The last one is used whether the given two pointers are pointing to the same address.

Pointer Arithmetic

```c
void main(){
  char s[] = "abcd";
  char *p1 = s;
  for(int i=0; i<4; i++){
      printf("%c," ,*p1);
      p1++;
  }
  printf("\n");
  int a = 97, b = 43;
  int *p2 = &a, *p3 = &b;
  printf("p2: %p\n",p2);
  printf("p3: %p\n",p3);
  printf("p2-p3: %d\n",p2-p3);
  int *p4 = &b;
  printf("p2 == p3: %d\n",p2 == p3);
  printf("p3 == p4: %d\n",p3 == p4);
  printf("p2 > p3: %d\n",p2 > p3);
}
```

Output:

a,b,c,d,
p2: 00000042b81ffcfc
p3: 00000042b81ffcf8
p2-p3: 1
p2 == p3: 0
p3 == p4: 1
p2 > p3: 1

Exercises

Solve the following by using pointers.

1. Write a function that traverses an array's elements and prints them.
2. Write a function that traverses a matrix's elements and prints them in a matrix format.
3. Write a function that finds the length of a string by pointer arithmetic.
4. Write a function that capitalizes the given string.
5. Write a function that reverses the given string.

Table of Contents

1 Pointers

2 Pointers on Stack

3 Pointers on Heap

Heap

- Stack has memory shortage issues compared to the heap.
- Stack is not flexible, so the allocated memory cannot be changed.
- Heap allows more flexible and efficient memory usage.
- The memory allocation is done dynamically on the heap. Therefore, the programmer can resize the allocated memory on the heap.
- Despite all the pros of the heap, accessing data on the heap is slower than the stack.

Heap

- We allocate memory to the heap...
  - when we need a lot of memory for our data,
  - when we don't know the size of a variable or its size dynamically changes during the execution of the program,
  - when we want the local variables declared in a scope to survive outside of its block.
  - when we want to define more complex structures such as linked lists, trees, graphs, etc.

Heap

- The following are the necessary functions in <**stdlib.h**> library to handle dynamic memory allocation to heap in C.
  - **malloc()**
  - **calloc()**
  - **realloc()**
  - **free()**
- The first three of them are used for memory allocation to the heap and the last one is used to free up this reserved memory space once we are done with it.

Heap

- **malloc()** (memory allocation) allocates a specified amount of bytes to the heap.
- **calloc()** (contiguous allocation) also allocates a specified amount of bytes to the heap. It initializes all bits to zero.
- **realloc()** (re allocation) changes the size of the memory space of the current memory allocation. If the new size exceeds the current, it protects the current data.
- **free()** deallocates the memory space declared by the functions above.

malloc() and free()

void* malloc(byte_size)

- malloc returns a void pointer, which needs to be typecasted.
- After we are done with the pointer allocated to the heap, we better deallocate it to the heap so that we can free that space to be used later by the program.

void free(pointer)

malloc() and free()

```
1  #include<stdlib.h>
2
3  void main(){
4   int *p = (int *)malloc(sizeof(int));
5   double *q = (double *)malloc(sizeof(double));
6   int *r = (int *)malloc(sizeof(int)*10);
7   int (*mtrx)[5] = malloc(3*sizeof(*mtrx));
8   free(p);
9   free(q);
10  free(r);
11  free(mtrx);
12 }
```

malloc() and free()

- In case of any memory access violation on the heap, it is better to check whether the declared pointer is NULL after its declaration.

```
1 #include<stdlib.h>
2
3 void main(){
4  int *p = (int *)malloc(1000 * sizeof(int));
5  if (p == NULL){
6   printf("Pointer could not be declared!\n");
7  } else {
8   for(int i=0; i<1000; i++)
9     p[i] = i;
10 }
11 free(p);
12 }
```

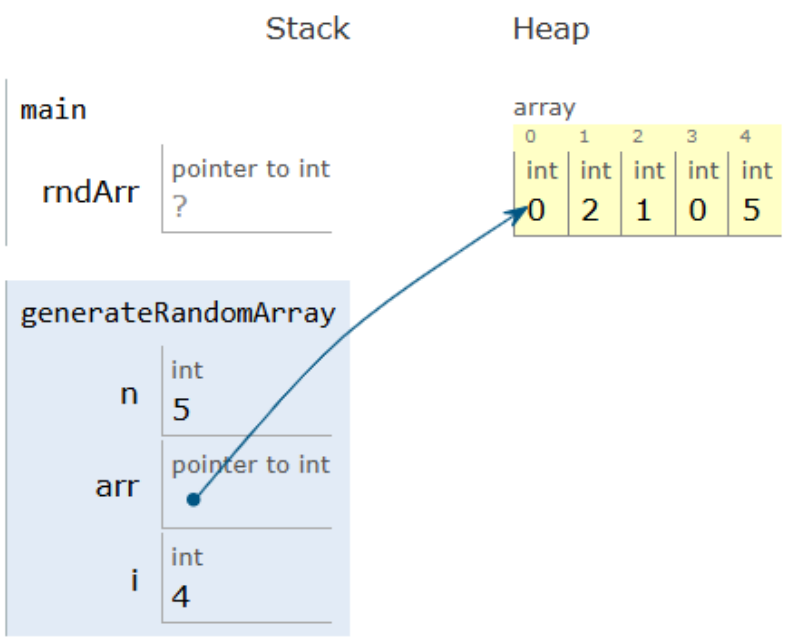
malloc() and free()

C (C17 + GNU extensions)
known limitations

```c
void * generateRandomArray(int n){
    int *arr = malloc(n*sizeof(int));
    for(int i=0; i<n; i++){
        arr[i] = rand() % 10;
    }
    return arr;
}

int main() {
    srand(time(0));
    int * rndArr = generateRandomArray(5);
    for(int i=0; i<5; i++){
        printf("%d, ",rndArr[i]);
    }
    return 0;
}
```

Print output (drag lower right corner to resize)



Note: ? refers to an uninitialized value

Figure: https://pythontutor.com/render.html#mode=display

malloc() and free()

C (C17 + GNU extensions)
known limitations

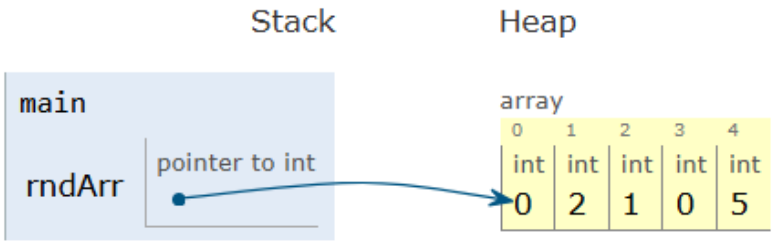
```c
void * generateRandomArray(int n){
    int *arr = malloc(n*sizeof(int));
    for(int i=0; i<n; i++){
        arr[i] = rand() % 10;
    }
    return arr;
}

int main() {
    srand(time(0));
    int * rndArr = generateRandomArray(5);
    for(int i=0; i<5; i++){
        printf("%d, ",rndArr[i]);
    }
    return 0;
}
```

Edit this code

Print output (drag lower right corner to resize)

```
0, 2, 1, 0, 5,
```



C/C++ details: none [default view]

Figure: https://pythontutor.com/render.html#mode=display

Exercises

Solve the following by using pointers allocated to the heap.

1. Write a function that receives an array and its size as its parameters and returns the min, max, and sum of the given array via the help of a pointer.
2. Write a function that ciphers a string with ceasar cipher using heap allocation.

References

Deitel, P. and Deitel, H. (2022). *C How to Program*. Pearson, 9th edition.